委託計算を用いたFHE-TFHEスキームスイッチングの性能評価

ピウソダリ　ソンビラサイ[1,a)]　千田　浩司[1]

概要：整数加減乗算に適したBFVや近似演算に適したCKKSと，ブール演算に適したTFHEを組み合わせたハイブリッド準同型暗号では，CKKS暗号文からTFHE暗号文へのスキームスイッチングが重要な役割を果たす．既存方式では，変換前にスキームスイッチング用評価鍵の生成や事前計算が必要であり，大きなセットアップコストを要する．本研究では，これらの事前計算を用いず，変換処理の一部をProxyへ委託するFHE-TFHE変換フレームワークを検討する．その具体例として，マスキングとServer-Proxy間の閾値復号を用いたBFV-TFHEスキームスイッチング方式を提案する．提案方式では，マスキングと閾値復号によって入力を秘匿し，TFHEマスク除去回路を用いる方式と，ABYによるA2B変換を用いる方式の2方式でTFHE暗号文を生成する．OpenFHEおよびABYを用いて両方式を実装し，セットアップ時間と変換処理時間を評価する．

キーワード：FHE，TFHE，スキームスイッチング

Performance Evaluation of FHE-TFHE Scheme Switching Using Outsourced Computation

Songvilasay Phiousodalith[1,a)]　Koji Chida[1]

Abstract: In hybrid homomorphic encryption systems that combine BFV, which is suitable for integer addition, subtraction, and multiplication, or CKKS, which is suitable for approximate arithmetic, with TFHE, which is suitable for Boolean operations, scheme switching from CKKS ciphertexts to TFHE ciphertexts plays an important role. Existing methods require the generation of scheme-switching evaluation keys and precomputation before conversion, resulting in significant setup costs. This study investigates an FHE-to-TFHE conversion framework that delegates part of the conversion process to a proxy without requiring such precomputation. As a concrete example, we propose a BFV-to-TFHE scheme-switching method based on masking and threshold decryption between a server and a proxy. The proposed method protects the input through masking and threshold decryption and generates TFHE ciphertexts using two approaches: a TFHE-based mask-removal circuit and an A2B conversion using ABY. We implement both approaches using OpenFHE and ABY and evaluate their setup and conversion times.

Keywords: FHE，TFHE，Scheme Switching

1. はじめに

近年，クラウドコンピューティングの普及により，利用者が自身のデータを外部のサーバへ送信し，その計算資源を利用してデータ処理を行う委託計算が広く利用されている．委託計算を利用することで，利用者自身が大規模な計算環境を用意することなく，高性能な計算資源を利用できる．一方，医療情報や個人情報などの機密性の高いデータを外部のサーバで処理する場合，計算を実行するためにデータを復号すると，サーバ側から平文データが参照さ

[1] 群馬大学情報学部, 371-8510 群馬県前橋市荒牧町 4-2,
Faculty of Informatics, Gunma University, 4-2, Aramakicho, Maebashi, Gunma, Japan
[a)] j251a021@gunma-u.ac.jp

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

れる可能性がある．通常の暗号化では，保存時や通信時のデータを保護することはできるが，暗号化された状態のままデータを計算することは困難である．

このような問題に対し，完全準同型暗号（Fully Homomorphic Encryption: FHE）は，暗号化されたデータを復号することなく演算できる技術であり，プライバシーを保護したまま外部の計算資源を利用できる技術として研究されている [1]．一方，FHE 方式はそれぞれ得意とする演算が異なる．例えば，CKKS 方式は実数・複素数に対する近似算術演算を効率的に実行するために提案された方式である [2]. これに対して，FHEW および TFHE は高速なブートストラップを利用した論理演算に適した方式である [3], [4]．そのため，数値演算と比較・条件分岐などの異なる種類の処理を組み合わせる場合，一つの FHE 方式だけですべての処理を効率的に実行するのではなく，各方式の特徴を組み合わせて利用することが考えられる．このようなハイブリッドな暗号化計算を実現するためには，計算途中で暗号方式を切り替えるスキームスイッチングが重要となる．

OpenFHE は, CKKS, BFV[5], BGV[6], FHEW/TFHE など複数の FHE 方式をサポートするオープンソースライブラリであり, 異なる暗号方式を組み合わせた処理も提供している [7], [8]．特に，CKKS 暗号文から FHEW/BinFHE 暗号文へのスキームスイッチングを利用することで，CKKS による数値演算と FHEW/TFHE 系のブートストラップを用いた比較・非線形処理を組み合わせることができる．一方，OpenFHE の CKKS から FHEW/BinFHE への直接変換では，変換処理に先立ってスキームスイッチング用の鍵生成，ブートストラップ鍵生成，および事前計算などの準備処理が必要となる．

本稿では，TFHE 以外の FHE 暗号文から TFHE 暗号文への変換処理の一部を複数の計算主体へ委託するスキームスイッチングフレームワークを検討する．本フレームワークでは，Client，Server，Proxy の 3 つの論理的な役割を想定し，通常の準同型演算は Server が実行し，スキームスイッチングが必要となる場合にのみ Proxy と協調して変換を行う．Server と Proxy が結託しない限り，Client の秘密情報の秘匿性は保たれる．本稿では，このフレームワークの具体例として BFV 暗号文から TFHE 暗号文への変換を対象とする．提案方式では，Server が BFV 暗号文にランダム値によるマスキングを行い，Server と Proxy による閾値復号によって，Proxy がマスクされた値を取得する．主方式では，Proxy がこの値をビット分解し，各ビットを TFHE 暗号化して Server へ送信する．Server は自身が保持するマスクを用いて TFHE 加算路回を評価することで，元の入力値に対応する TFHE 暗号文を生成する．また，別の構成として，Server と Proxy が保持する加法的シェアに対して，2 者間 MPC による Arithmetic-to-Boolean（A2B）変換を行い，得られた Boolean シェアから TFHE 暗号文を生成する方式についても検討する [9], [10]．本稿では，これら 2 方式を実装し，性能比較を行う．

本節の構成は，次のとおり．次節で，提案手法に利用される準備知識について簡単に説明する． 3 節で，既存方法について説明する． 4 節では，提案フレームワークについて説明する． 5 節では，提案手法の実験結果を表示し， 6 節で，まとめる．

2. 準備

2.1 完全準同型暗号

完全準同型暗号（Fully Homomorphic Encryption: FHE）は，データを復号することなく，暗号化された状態のまま加算や乗算などの演算を実行できる暗号方式である [1]．平文 m を暗号化した暗号文を $ct = \mathrm{Enc}(m)$ とすると，関数 f に対応する準同型演算を暗号文上で実行することで，$f(m)$ に対応する暗号文を得ることができる．

$$\mathrm{Dec}\left(\mathrm{Eval}(f, \mathrm{Enc}(m))\right) = f(m)$$

FHE には，扱うデータや演算に応じて異なる方式が存在する．例えば，CKKS は実数・複素数に対する近似数値演算に適しており，TFHE はビット単位の論理演算に適している．

2.2 BFV 方式

BFV（Brakerski–Fan–Vercauteren）方式は，整数に対する加算および乗算を暗号文上で実行できる準同型暗号方式である [5]．BFV の平文空間は，一般に

$$R_t = \mathbb{Z}_t[X]/(X^N + 1)$$

として定義される．ここで，N は多項式環の次数，t は平文法を表す．

BFV では，入力整数ベクトル

$$\boldsymbol{m} = (m_0, m_1, \ldots, m_{\ell-1}), \qquad m_i \in \mathbb{Z}_t$$

を平文多項式へ符号化して暗号化する．また，バッチ符号化を用いることで，複数の整数を一つの暗号文の異なるスロットに格納し，各スロットに対して並列に演算できる．

二つの暗号文 $ct_1 = \mathrm{Enc}(m_1)$ および $ct_2 = \mathrm{Enc}(m_2)$ に対して準同型演算を行うと，

$$\mathrm{Dec}(ct_1 + ct_2) = m_1 + m_2 \pmod t,$$

$$\mathrm{Dec}(ct_1 \cdot ct_2) = m_1 m_2 \pmod t$$

が得られる．暗号文ノイズが復号可能範囲内であれば，演算結果を正しく復号できる．

© 2026 Information Processing Society of Japan

2.3 TFHE 方式と OpenFHE BinFHE

TFHE（Fully Homomorphic Encryption over the Torus）は，LWE 暗号を基礎とし，暗号化されたビットに対する論理演算を効率的に実行する準同型暗号方式である [4]. TFHE で用いられる LWE 暗号文は，$\boldsymbol{c} = (\boldsymbol{a}, b) \in \mathbb{Z}_q^n \times \mathbb{Z}_q$ と表される．秘密鍵を $\boldsymbol{s} \in \mathbb{Z}_q^n$ とすると，平文 $m \in \mathbb{Z}_p$ に対する暗号文の第 2 成分は

$$b = \langle \boldsymbol{a}, \boldsymbol{s} \rangle + \left\lfloor \frac{q}{p} \right\rceil m + e \pmod{q}$$

によって生成される．ここで，$\boldsymbol{a}$ は一様ランダムに選択されるベクトル，e は小さな誤差，p および q はそれぞれ平文モジュラスと暗号文モジュラスである．復号時には

$$\phi = b - \langle \boldsymbol{a}, \boldsymbol{s} \rangle \pmod{q}$$

を計算し，ϕ に最も近い平文表現へ丸めることで m を復元する．特に Boolean 値を扱う場合は $m \in \{0, 1\}$ となる．

暗号文に対して演算を繰り返すと誤差が増大し，正しい復号が困難になる．TFHE では，ブートストラップ鍵を用いて暗号文を復号せずに誤差を低減するブートストラッピングを実行する．また，ブートストラッピングと論理関数の評価を組み合わせることにより，入力暗号文を更新すると同時に，暗号化されたビットに対して AND，OR，XOR，NAND などの論理ゲートを評価できる．これにより，多数の論理ゲートから構成される Boolean 回路を暗号文上で継続的に評価できる．

TFHE 系暗号の実装として OpenFHE ライブラリの BinFHE モジュールを使用する．BinFHE は，FHEW および TFHE 系のブートストラッピング方式に基づく Boolean 回路評価用のインタフェースである [3], [7].

2.4 閾値復号

閾値復号（Threshold Decryption）は，暗号文の復号に必要な秘密鍵を複数の参加者に分散して保持し，複数者が協調することで復号を行う方式である．通常の公開鍵暗号では，一つの秘密鍵 sk を用いて

$$m = \mathrm{Dec}_{sk}(ct)$$

として暗号文 ct を復号する．一方，閾値復号では秘密鍵に対応する情報を複数の参加者に分散して保持するため，一人の参加者のみでは暗号文を復号できない．

例えば，2 者 P_1 および P_2 による閾値復号を考える．各参加者が秘密鍵シェア sk_1，sk_2 を保持するとき，それぞれが暗号文 ct に対して分散復号を行い，分散復号結果

$$d_1 = \mathrm{PartialDec}_{sk_1}(ct), \qquad d_2 = \mathrm{PartialDec}_{sk_2}(ct)$$

を生成する．その後，これらの分散復号結果を統合することで，

$$m = \mathrm{Fusion}(d_1, d_2)$$

として元の平文を復元する．

このように，閾値復号では秘密鍵を一つの計算主体に集中させることなく，複数者による協調によって暗号文を復号できる．

2.5 加法的秘密分散

加法的秘密分散は，秘密値を複数のシェアに分割し，各シェア単独からは元の値を復元できないようにする手法である [11]．本稿では，k ビット整数を扱うため，秘密値およびシェアを剰余環

$$\mathbb{Z}_{2^k} = \{0, 1, \ldots, 2^k - 1\}$$

の要素として表現する．二者 Alice および Bob が秘密値 $m \in \mathbb{Z}_{2^k}$ を共有する場合，Alice のシェアを $[m]_A$，Bob のシェアを $[m]_B$ とし，

$$m = [m]_A + [m]_B \pmod{2^k}$$

を満たすように生成する．

具体的には，Alice のシェア r を $\mathbb{Z}_{2^k}$ から一様ランダムに選び，$[m]_A = r$ とする．Bob のシェアは $[m]_B = m - r \pmod{2^k}$ として計算される．このとき，両者のシェアを加算すると，

$$[m]_A + [m]_B = r + (m - r) = m \pmod{2^k}$$

となり，元の秘密値を復元できる．一方，r が一様ランダムに選ばれる場合，各シェアを単独で観測しても m を特定することはできない．

加法的シェアに対する加算や公開定数倍は，各パーティが自身のシェアを局所的に計算するだけで実行できる．一方，比較，ビット抽出，論理演算などを行うためには，秘密値をビット単位で表現する必要があるため，加法的シェアをそのまま利用することはできない．そこで，加法的シェアとして保持された秘密値を，各ビットの Boolean シェアへ安全に変換する Arithmetic-to-Boolean（A2B）変換が必要となる．A2B 変換の詳細については，次で説明する．

2.6 Arithmetic-to-Boolean 変換

Arithmetic-to-Boolean（A2B）変換は，算術シェアとして保持された秘密整数を，各ビットの Boolean シェアへ安全に変換する MPC プロトコルである [9], [10]．k ビット整数 $m \in \mathbb{Z}_{2^k}$ に対する二者間の算術シェアを

$$m = [m]_A + [m]_B \pmod{2^k}$$

とする．A2B 変換では，これを

$$m = \sum_{j=0}^{k-1} 2^j m_j, \qquad m_j = \langle m_j \rangle_A \oplus \langle m_j \rangle_B$$

© 2026 Information Processing Society of Japan

を満たす各ビットの Boolean シェアへ変換する．ここで，$\langle m_j \rangle_A$ および $\langle m_j \rangle_B$ は Alice と Bob がそれぞれ保持する Boolean シェアである．したがって，変換後も各パーティが m_j を単独で知ることはできない．

算術シェアを各パーティが個別にビット分解し，対応するビットを単純に XOR するだけでは正しいビット表現は得られない．これは，算術シェアの加算では下位ビットから上位ビットへの桁上がりが発生するためである．例えば，

$$[m]_A = 7 = (0111)_2, \qquad [m]_B = 2 = (0010)_2$$

の場合，

$$m = 7 + 2 = 9 = (1001)_2$$

であるのに対し，

$$0111 \oplus 0010 = 0101$$

となり，正しいビット列とは一致しない．したがって，A2B 変換では桁上がりを含む加算処理を秘密分散された状態で安全に実行する必要がある．

A2B 変換を実現するフレームワークとして，ABY が提案されている [9]．ABY は 2 者間セキュア計算のためのフレームワークであり，Arithmetic Sharing，Boolean Sharing，Yao Sharing などの異なる秘密分散表現を組み合わせて計算できる．また，各 Sharing 間の変換機能を提供しており，Arithmetic Sharing で表現された秘密値を，秘密値を明かすことなく Boolean Sharing へ変換できる．本研究では，Server と Proxy が保持する加法的シェアに対して，ABY を用いた A2B 変換を行い，各ビットに対応する Boolean シェアを生成する．

具体的には，BFV 暗号文に対するマスキングおよび閾値復号により，第 i スロットの整数 m_i に対する算術シェア

$$[m_i]_A + [m_i]_B = m_i \pmod{2^k}$$

を満たすように生成する．その後，A2B 変換により，これらの算術シェアを各ビットの Boolean シェアへ変換する．

$$([m_i]_A, [m_i]_B) \xrightarrow{\text{A2B}} \left(\{\langle m_{i,j} \rangle_A\}_{j=0}^{k-1}, \{\langle m_{i,j} \rangle_B\}_{j=0}^{k-1}\right).$$

ここで，Alice は $(m_{i,j})_A$ を，Bob は $(m_{i,j})_B$ を保持し，各ビット j について

$$m_{i,j} = \langle m_{i,j} \rangle_A \oplus \langle m_{i,j} \rangle_B$$

が成立する．したがって，A2B 変換後も平文ビット $m_{i,j}$ はいずれのパーティにも復元されない．

3. 既存手法による CKKS-TFHE スキームスイッチング

CKKS は実数・複素数に対する近似的な算術演算を効率的に実行できる一方，TFHE はビット単位の論理演算や比較演算に適している．そのため，CKKS による算術演算の結果に対して比較や条件分岐などを適用する場合，CKKS 暗号文を TFHE 暗号文へ変換するスキームスイッチングが必要となる [8]．既存手法として OpenFHE[7] が提供する CKKS から BinFHE へのスキームスイッチングを用いる．この方式は，大きく変換前のセットアップ処理と，CKKS 暗号文を BinFHE 暗号文へ変換する処理から構成される．

3.1 セットアップ処理

CKKS から BinFHE への変換を実行する前に，スキームスイッチングに必要となる鍵およびパラメータを準備する．具体的には，まず EvalCKKStoFHEWSetup により CKKS-BinFHE 間のスキームスイッチングに必要な環境を生成する．次に，EvalCKKStoFHEWKeyGen により変換に必要な評価鍵を生成する．また，BinFHE 側では BTKeyGen によりブートストラップ鍵を生成する．さらに，EvalCKKStoFHEWPrecompute により CKKS 暗号文から BinFHE 暗号文への変換に必要な事前計算を行う．したがって，既存方式ではオンライン変換を実行する前に，Setup → Evaluation KeyGen → Bootstrapping KeyGen → Precomputation の処理を行う必要がある．これらは基本的に変換前に実行される一時的なセットアップ処理であり，一度生成された鍵や事前計算結果は後続の変換処理で利用できる．

3.2 CKKS から BinFHE への変換

セットアップ完了後，CKKS 暗号文 $\mathrm{Enc}_{\mathrm{CKKS}}(m)$ を入力として，EvalCKKStoFHEW により BinFHE 暗号文への変換を行う．変換後，各 CKKS スロットに対応する BinFHE 暗号文 $\mathrm{Enc}_{\mathrm{BinFHE}}(\widetilde{m}_i)$ が得られる．ここで，$(\widetilde{m}_i)$ は CKKS 暗号文の第 (i) スロットに格納された値に対応する変換後の整数値を表す．本方式では，変換サーバに CKKS 秘密鍵および BinFHE 秘密鍵を渡すことなく，事前に生成された評価鍵を用いて暗号文間の変換を実行できる．一方，この方式では処理を実行する前に，スキームスイッチング専用の評価鍵生成および事前計算が必要となる．今回の実験環境を表 3 に示し，実験パラメータと実験結果を表 1 と 2 に示す．

通常の暗号化データに対する計算では，CKKS や TFHE を用いることで，Server が単独で準同型演算を実行できる．一方，CKKS から TFHE への変換処理を含む暗号化データ処理については，必ずしも Server 単独で完結させる必要はなく，別の計算主体に処理の一部を委託する構成が考えられる．そこで本研究では，通常の準同型演算は Server が単独で実行し，スキームスイッチングが必要となる場合にのみ Proxy と協調する委託計算モデルを検討する．

© 2026 Information Processing Society of Japan

分類	設定値
CKKS Security Level	HEStd_128_classic
BinFHE Security parameter set	STD128
入力数	100

表 1 既存方式の実装パラメータ

セットアップ時間 [s]	オンライン時間 [s]	合計時間 [s]
86.435	0.306	86.741

表 2 CKKS-TFHE スキームスイッチングの処理時間

4. 提案手法

4.1 システムモデル

本研究では，TFHE 以外の FHE 暗号文から TFHE 暗号文への変換処理の一部を Proxy へ委託するスキームスイッチングフレームワークを想定する． 本フレームワークでは，BFV，BGV，CKKS などの FHE 方式で暗号化されたデータに対して Server が準同型演算を実行し， TFHE によるビット単位の演算が必要となった場合に， Proxy と協調して TFHE 暗号文への変換を行う． 変換対象となる FHE 方式が閾値復号をサポートすることを前提とし， マスキングと閾値復号を利用することで，変換処理を Server と Proxy に分散する． 本研究では，このフレームワークの具体例として， 整数値演算に適した BFV から， ビット単位の演算に適した TFHE へのスキームスイッチングを対象とする． 以下では，Client，Server，および Proxy の 3 者から構成される，(TFHE 以外の)FHE-TFHE 変換システムについて説明する．

- **Client**：秘密情報である平文 $\boldsymbol{m}$ を保持するユーザである．Client は (TFHE 以外の)FHE 公開鍵を用いて平文 $\boldsymbol{m}$ を暗号化し，FHE 暗号文を生成して Server へ送信する．また，TFHE による準同型演算に必要なブートストラップ鍵（Bootstrapping Key）および評価鍵（Evaluation Key）を生成し，Server および Proxy へ必要な鍵情報を配布する．Client はこれらの初期処理を行った後，Server と Proxy によるスキームスイッチング処理には関与しない．
- **Server**：Client から受信した暗号文に対して，TFHE 以外の FHE または TFHE を用いた準同型演算を実行する主たる計算主体である．FHE から TFHE へのスキームスイッチングが必要となった場合には，ランダムマスク $\boldsymbol{r}$ を生成して FHE 暗号文をマスクし，Proxy と協調して FHE 閾値復号を実行する．A2B 変換を用いる方式では，Proxy とセキュア 2 パーティ計算を実行して算術シェアを Boolean シェアへ変換し，TFHE 暗号文の生成に参加する．一方，TFHE マスク除去回路を用いる方式では，自身が保持するマスク $\boldsymbol{r}$ を用いて TFHE 上でマスク除去のための回路を評価する．
- **Proxy**：Server から委託されたスキームスイッチング処理を補助する計算主体である．Proxy は通常の FHE および TFHE による準同型演算には参加せず，スキームスイッチングが必要となる場合に Server と協調して FHE 閾値復号を実行する．A2B 変換を用いる方式では，Server とセキュア 2 パーティ計算を実行し，Boolean シェアへの変換および TFHE 暗号文の生成に参加する．TFHE マスク除去回路を用いる方式では，閾値復号によって得られたマスク済みの値をビット分解し，各ビットを TFHE 方式により暗号化して Server へ送信する．

4.2 セキュリティ仮定

本研究では，Client，Proxy が Semi-Honest（Honest-but-Curious）モデルに従うものと仮定する．すなわち，各主体はプロトコルを正しく実行するが，実行中に取得した情報から平文を推測しようとする可能性がある．また，各主体は単独では平文を復元できず，秘密情報は暗号文または秘密分散された状態でのみ扱われることを前提とする．

4.3 提案プロトコル

本小節では，委託計算を用いた BFV-TFHE スキームスイッチングプロトコルについて説明する．提案プロトコルでは，Server と Proxy が協調して BFV 暗号文から TFHE 暗号文への変換を行う．本研究では，TFHE 暗号文への変換方法として 2 方式を検討する．以下では，まず TFHE マスク除去回路を用いる基本方式を示し，続いて A2B 変換を用いる方式について説明する．

4.3.1 TFHE マスク除去回路を用いる方式

本方式では，Server が BFV 暗号文にランダムマスクを適用し，Server と Proxy による閾値復号によってマスクされた整数を取得する．Proxy は得られた整数をビット分解し，各ビットを TFHE 暗号化して Server へ送信する．その後，Server は自身が保持するマスクを用いて TFHE 上で加算回路を評価し，元の入力整数に対応する TFHE 暗号文を生成する．以下に処理手順を示す．

BFV の平文法を $t = 2^K$ とし，すべての平文演算を $\mathbb{Z}_{2^K}$ 上で行うものとする．また，入力値 m_i は

$$-2^{K-1} \leq m_i < 2^{K-1}$$

を満たす K ビット符号付き整数とし，BFV 上では

$$M_i = m_i \bmod 2^K$$

によって得られる K ビット 2 の補数表現として扱う．

4.3.1.1 Step 1：BFV 暗号化

Client は入力ベクトル $\boldsymbol{m} = (m_0, m_1, \ldots, m_{n-1})$ を K ビット 2 の補数表現へ変換し，

$$\boldsymbol{M} = (M_0, M_1, \ldots, M_{n-1}) \in \mathbb{Z}_{2^K}^n$$

© 2026 Information Processing Society of Japan

を得る．ここで，各要素は $M_i = m_i \bmod 2^K$ によって求められる．Client は $\boldsymbol{M}$ を BFV 方式により暗号化し，BFV 暗号文

$$ct_{\mathrm{BFV}} = \mathrm{Enc}_{pk_{\mathrm{BFV}}}(\boldsymbol{M})$$

を生成して Server へ送信する．BFV 秘密鍵は Server と Proxy の間で分散して保持され，いずれか一方のみでは BFV 暗号文を復号できないものとする．

4.3.1.2 Step 2：マスキング

Server は，各入力値 M_i に対してランダムな整数マスク $R_i \leftarrow \mathbb{Z}_{2^K}$ を生成する．マスクベクトルを

$$\boldsymbol{R} = (R_0, R_1, \ldots, R_{n-1})$$

とする．Server は $\boldsymbol{R}$ を BFV で暗号化し，

$$ct_{\boldsymbol{R}} = \mathrm{Enc}_{pk_{\mathrm{BFV}}}(\boldsymbol{R})$$

を生成する．その後，BFV の加法準同型性を用いて，入力暗号文からマスク暗号文を減算して

$$ct_{\mathrm{mask}} = ct_{\mathrm{BFV}} - ct_{\boldsymbol{R}}$$

を得る．これにより，ct_{mask} の各要素については，

$$D_i = (M_i - R_i) \bmod 2^K$$

である．整数マスク $\boldsymbol{R}$ は Server が保持し，後の TFHE 加算回路によるマスク除去に利用する．

4.3.1.3 Step 3：BFV 閾値復号

Server と Proxy は，それぞれが保持する BFV 秘密鍵シェアを用いて，ct_{mask} に対する分散復号を行う．Server および Proxy の分散復号結果をそれぞれ pd_{Server} および pd_{Proxy} とすると，

$$pd_{\mathrm{Server}} = \mathrm{PartialDec}_{sk_{\mathrm{Server}}}(ct_{\mathrm{mask}})$$

および

$$pd_{\mathrm{Proxy}} = \mathrm{PartialDec}_{sk_{\mathrm{Proxy}}}(ct_{\mathrm{mask}})$$

となる．Proxy はこれらの分散復号結果を統合することで，マスクされた整数ベクトル $\boldsymbol{D} = (\boldsymbol{M} - \boldsymbol{R}) \bmod 2^K$ を取得する．

4.3.1.4 Step 4：Proxy による TFHE 暗号化

Proxy は D_i を K ビットに分解し，

$$D_i = \sum_{j=0}^{K-1} 2^j d_{i,j}$$

として表現する．ここで，$d_{i,j} \in \{0,1\}$ は D_i の j ビット目を表す．ビット列としては，$D_i = (d_{i,K-1}, \ldots, d_{i,1}, d_{i,0})$ と表される．Proxy は各ビット $d_{i,j}$ を TFHE 方式によって暗号化し，

$$ct_{i,j}^{(d)} = \mathrm{Enc}_{pk_{\mathrm{TFHE}}}(d_{i,j})$$

を生成して Server へ送信する．このビット分解は Proxy がローカルに実行する．また，Server が受信するのは D_i の各ビットに対応する TFHE 暗号文であり，Server は D_i そのものを平文として取得しない．

4.3.1.5 Step 5：TFHE 加算回路によるマスク除去

Server は，自身が保持する整数マスク R_i と，Proxy から受信した D_i の TFHE 暗号文を用いてマスクを除去する．Server は，D_i の暗号化された各ビットと R_i の平文ビットを入力として，TFHE 上でリップルキャリー加算回路を評価する．R_i を

$$R_i = \sum_{j=0}^{K-1} 2^j r_{i,j}$$

と表す．ここで，$r_{i,j} \in \{0,1\}$ は R_i の j ビット目である．$d_{i,j}$ を D_i の j ビット目，$r_{i,j}$ を R_i の j ビット目，$c_{i,j}$ を j ビット目へのキャリーとする．最下位ビットへの初期キャリーは $c_{i,0} = 0$ とする．一般的な 1 ビット全加算器では，出力ビット $m_{i,j}$ および次のビットへのキャリー $c_{i,j+1}$ は，

$$m_{i,j} = d_{i,j} \oplus r_{i,j} \oplus c_{i,j}$$

および

$$c_{i,j+1} = (d_{i,j} \wedge r_{i,j}) \vee (d_{i,j} \wedge c_{i,j}) \vee (r_{i,j} \wedge c_{i,j})$$

として求められる．ここで，$r_{i,j}$ は Server が保持するマスクのビットであり，暗号化された値ではなく既知の平文ビットである．したがって，$r_{i,j}$ の値に応じて TFHE 加算回路を次のように簡略化できる．

$r_{i,j} = 0$ の場合，

$$\begin{cases} m_{i,j} = d_{i,j} \oplus c_{i,j}, \\ c_{i,j+1} = d_{i,j} \wedge c_{i,j}. \end{cases}$$

一方，$r_{i,j} = 1$ の場合，

$$\begin{cases} m_{i,j} = \neg(d_{i,j} \oplus c_{i,j}), \\ c_{i,j+1} = d_{i,j} \vee c_{i,j}. \end{cases}$$

Server は，Proxy から受信した $ct_{i,j}^{(d)}$ と暗号化されたキャリーを用いて，上記の XOR，AND，OR および NOT ゲートを TFHE 上で評価する．この処理を最下位ビットから最上位ビットまで順に実行し，各ビットで得られた暗号化キャリーを次のビットの入力とする．これにより，$M_i = (D_i + R_i) \bmod 2^K$ を暗号化状態のまま計算できる．2^K を法とする加算であるため，最上位ビットから生じるキャリー $c_{i,K}$ は破棄する．最終的に，各ビットについて

$$ct_{i,j}^{(m)} = \mathrm{Enc}_{pk_{\mathrm{TFHE}}}(m_{i,j})$$

を得る．

© 2026 Information Processing Society of Japan

4.3.2 A2B 変換を用いる方式

本方式では，TFHE 加算回路を用いる方式とは異なり，Server と Proxy が保持する加法的シェアを 2 者間 MPC によって Boolean シェアへ変換し，それぞれの Boolean シェアから TFHE 暗号文を生成する．Client による BFV 暗号化，Server によるマスキング，および BFV 閾値復号は前方式と同様であるため省略する．閾値復号後，Server はランダムマスク R_i を保持し，Proxy はマスクされた整数

$$D_i = (M_i - R_i) \bmod 2^K$$

を保持する．以下では，

$$A_i = R_i, \qquad B_i = D_i$$

とする．これらは，$(A_i + B_i) \bmod 2^K = M_i$ を満たすため，A_i と B_i を元の入力値 M_i に対する加法的シェアとして扱うことができる．

4.3.2.1 Step 4：A2B 変換

Server と Proxy は，それぞれが保持する加法的シェア A_i および B_i を 2 者間 MPC へ秘密入力として与え，A2B（Arithmetic-to-Boolean）変換を実行する．本実装では ABY を用いて，$M_i = (A_i + B_i) \bmod 2^K$ を K 個のビットに対応する Boolean シェアへ変換する．M_i を

$$M_i = \sum_{j=0}^{K-1} 2^j x_{i,j}$$

と表す．ここで，$x_{i,j} \in \{0, 1\}$ は M_i の j ビット目である．A2B 変換後，Server は Boolean シェア $x_{i,j}^{(S)}$ を取得し，Proxy は Boolean シェア $x_{i,j}^{(P)}$ を取得する．これらは

$$x_{i,j} = x_{i,j}^{(S)} \oplus x_{i,j}^{(P)}$$

を満たす．A2B 変換は ABY の 2 者計算内部で実行されるため，M_i および各ビット $x_{i,j}$ は Server と Proxy のいずれにも平文として公開されない．

4.3.2.2 Step 5：TFHE 暗号文の生成

Proxy は，保持する Boolean シェアを TFHE 方式によって暗号化する．各ビットについて，

$$ct_{i,j}^{(P)} = \mathrm{Enc}_{pk_{\mathrm{TFHE}}}\left(x_{i,j}^{(P)}\right)$$

を生成し，Server に送信する．

その後，Server で TFHE 上で XOR ゲートを評価する．

$$ct_{i,j} = \mathrm{EvalXOR}\left(x_{i,j}^{(S)}, ct_{i,j}^{(P)}\right)$$

Boolean シェアの関係より，

$$\begin{aligned} ct_{i,j} &= \mathrm{Enc}_{pk_{\mathrm{TFHE}}}\left(x_{i,j}^{(S)} \oplus x_{i,j}^{(P)}\right) \\ &= \mathrm{Enc}_{pk_{\mathrm{TFHE}}}(x_{i,j}) \end{aligned}$$

となる．

5. 実装評価

5.1 実験環境

今回の実験環境を表 3 に示す．

OS	Ubuntu 24.04.3 LTS
実行環境	WSL2
Kernel	Linux 5.15.167.4-microsoft-standard-WSL2
Architecture	x86_64
CPU	Intel Core i5-1135G7 @ 2.40 GHz
Logical CPU	8
Memory	3.74 GiB
Compiler	GCC 13.3.0
C++ Standard	C++17
OpenFHE	1.5.1
MPC Framework	ABY

表 3 実験環境の概要

5.2 実験条件

提案方式を OpenFHE および ABY を用いて実装し，処理時間を比較した．主な実行パラメータを表 4 に示す．

項目	設定値
セキュリティレベル	128 bit
入力数	100
整数ビット幅 K	16 bit
BFV 閾値復号参加者数	2
BFV 平文法（TFHE 加算回路）	65537
BFV 平文法（A2B 方式）	2^{16}
TFHE パラメータセット	STD128
TFHE ブートストラップ方式	GINX
TFHE 加算回路	リップルキャリー加算
A2B 算術共有の法	2^{16}
A2B 出力共有	Boolean GMW Sharing

表 4 実験パラメータ

5.3 実行結果および分析

各方式のセットアップ時間およびオンライン処理時間を表 5 に示す．実験結果より，既存の CKKS-TFHE 方式のセットアップ時間が約 86.435 秒であったのに対し，A2B 方式では約 6.204 秒，TFHE マスク除去回路方式では約 2.759 秒となった．提案方式では，CKKS-TFHE スキームスイッチングで必要となる評価鍵生成および事前計算を使用しないため，セットアップ時間が短縮された．ただし，既存方式は CKKS-TFHE 変換，提案方式は BFV-TFHE 変換であり，暗号方式および処理条件が異なるため，厳密な性能比較ではなく，変換フレームワーク間の参考比較として位置付ける．オンライン処理時間は，既存方式の約 0.306 秒に対し，A2B 方式では約 19.277 秒，TFHE マスク除去回路方式では約 469.670 秒となった．A2B 方式では，A2B 変

© 2026 Information Processing Society of Japan

換に約 1.509 秒，Server 側と Proxy 側の TFHE 暗号化に合計約 17.768 秒を要しており，TFHE 暗号化が主要な処理時間を占めた．一方，TFHE マスク除去回路方式では，TFHE 暗号化に約 16.797 秒，リップルキャリー加算回路の評価に約 452.873 秒を要したため，TFHE 上のキャリー伝播処理が主要なボトルネックとなった．

以上より，今回測定した計算時間では，A2B 方式は TFHE マスク除去回路方式より約 24.4 倍高速であった．ただし，本実験では実ネットワーク上の通信時間を評価していない．A2B 方式では 2 者間 MPC による通信が必要となるため，今後は Server と Proxy を異なる計算機上に配置し，通信量および通信遅延を含めた評価を行う必要がある．一方，TFHE マスク除去回路方式を TFHE-rs[12], [13] で実装することで，処理時間を改善できる可能性がある．

方式	Setup [s]	Online [s]
A2B 方式	6.204	19.277
TFHE マスク除去回路方式	2.759	469.670

表 5　各方式の実行時間

実行者	処理	時間（s）
Proxy	マスク付き整数の TFHE 暗号化	16.797
Server	TFHE リップルキャリー加算回路	452.873

表 6　TFHE マスク除去回路方式における TFHE 処理時間

6. まとめ

本研究では，TFHE 以外の FHE 暗号文から TFHE 暗号文への変換処理の一部を Proxy へ委託するフレームワークを検討し，その具体例として BFV-TFHE スキームスイッチング方式を提案した．提案方式として，マスキングと閾値復号を用い，Proxy による TFHE 暗号化と Server による TFHE マスク除去回路によって変換する方式，および 2 者間 MPC による A2B 変換を用いる方式を実装した．評価の結果，提案した 2 方式は，既存の CKKS–TFHE 方式と比較してセットアップ時間を削減できることを確認した．ただし，暗号方式および実験条件が異なるため，本比較は変換フレームワーク間の参考比較として位置付けられる．

今後は，提案フレームワークを CKKS-TFHE 変換へ適用して実験するとともに，Server-Proxy 間の通信時間を含めた性能評価を行う予定である．また，複数の TFHE ライブラリ上で提案方式を実装・評価し，TFHE 暗号化およびゲート演算に適したライブラリを検討する．さらに，CHIMERA などの先行研究を実装し，提案方式との比較評価を行う．

謝辞　本研究は，JST 経済安全保障重要技術育成プログラム【JPMJKP24U5】の支援を受けたものです．

参考文献

[1] Craig Gentry. Fully homomorphic encryption using ideal lattices. In *Proceedings of the forty-first annual ACM symposium on Theory of computing*, pages 169–178, 2009.

[2] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song. Homomorphic encryption for arithmetic of approximate numbers. In *Advances in Cryptology – ASIACRYPT 2017*, volume 10624 of *Lecture Notes in Computer Science*, pages 409–437. Springer, 2017.

[3] Léo Ducas and Daniele Micciancio. FHEW: Bootstrapping homomorphic encryption in less than a second. In *Advances in Cryptology – EUROCRYPT 2015*, volume 9056 of *Lecture Notes in Computer Science*, pages 617–640. Springer, 2015.

[4] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. TFHE: Fast fully homomorphic encryption over the torus. *Journal of Cryptology*, 33(1):34–91, 2020.

[5] Junfeng Fan and Frederik Vercauteren. Somewhat practical fully homomorphic encryption. *Cryptology ePrint Archive*, 2012.

[6] Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. (leveled) fully homomorphic encryption without bootstrapping. *ACM Transactions on Computation Theory (TOCT)*, 6(3):1–36, 2014.

[7] OpenFHE Development Team. OpenFHE development repository. https://github.com/openfheorg/openfhe-development, 2026. Accessed: 2026-08-10.

[8] Ahmad Al Badawi, Andreea Alexandru, Jack Bates, Flavio Bergamaschi, David Bruce Cousins, Saroja Erabelli, Nicholas Genise, Shai Halevi, Hamish Hunt, Andrey Kim, Yongwoo Lee, Zeyu Liu, Daniele Micciancio, Carlo Pascoe, Yuriy Polyakov, Ian Quah, Saraswathy R.V., Kurt Rohloff, Jonathan Saylor, Dmitriy Suponitsky, Matthew Triplett, Vinod Vaikuntanathan, and Vincent Zucca. OpenFHE: Open-source fully homomorphic encryption library. Cryptology ePrint Archive, Paper 2022/915, 2022.

[9] Daniel Demmler, Thomas Schneider, and Michael Zohner. ABY—a framework for efficient mixed-protocol secure two-party computation. In *Proceedings of the 22nd Annual Network and Distributed System Security Symposium (NDSS)*. The Internet Society, 2015.

[10] Daniel Escudero, Satrajit Ghosh, Marcel Keller, Rahul Rachuri, and Peter Scholl. Improved primitives for mpc over mixed arithmetic-binary circuits. In *Advances in Cryptology – CRYPTO 2020*, volume 12172 of *Lecture Notes in Computer Science*, pages 823–852. Springer, 2020.

[11] Lizhi Xiong, Wenhao Zhou, Zhihua Xia, Qi Gu, and Jian Weng. Efficient privacy-preserving computation based on additive secret sharing, 2021.

[12] Zama. TFHE-rs: A pure rust implementation of the TFHE scheme for boolean and integer arithmetics over encrypted data. https://github.com/zama-ai/tfhe-rs, 2022. Accessed: 2026-08-21.

[13] Zama. TFHE-rs benchmarks. https://docs.zama.org/tfhe-rs/get-started/benchmarks, 2026. Accessed: 2026-08-21.

© 2026 Information Processing Society of Japan